

Abstract geometric lines in the top left corner, consisting of several thin, light brown lines forming a complex, overlapping pattern of polygons and triangles.

PERFORMANCE EVALUATION OF A SERVERLESS FAAS APPLICATION USING AZURE FUNCTIONS

Presented by:

- 1. Kavini Kushani**
- 2. Taniya Afreen**

INTRODUCTION

Cloud Computing

Scalable, on-demand computing resources without managing infrastructure.

Serverless Computing (FaaS)

Event-driven execution with automatic scaling and pay-per-use pricing.

Cloud Platforms

Azure Functions, AWS Lambda, and Google Cloud Functions.

Performance Challenges

Cold starts, response time variability, and scalability.

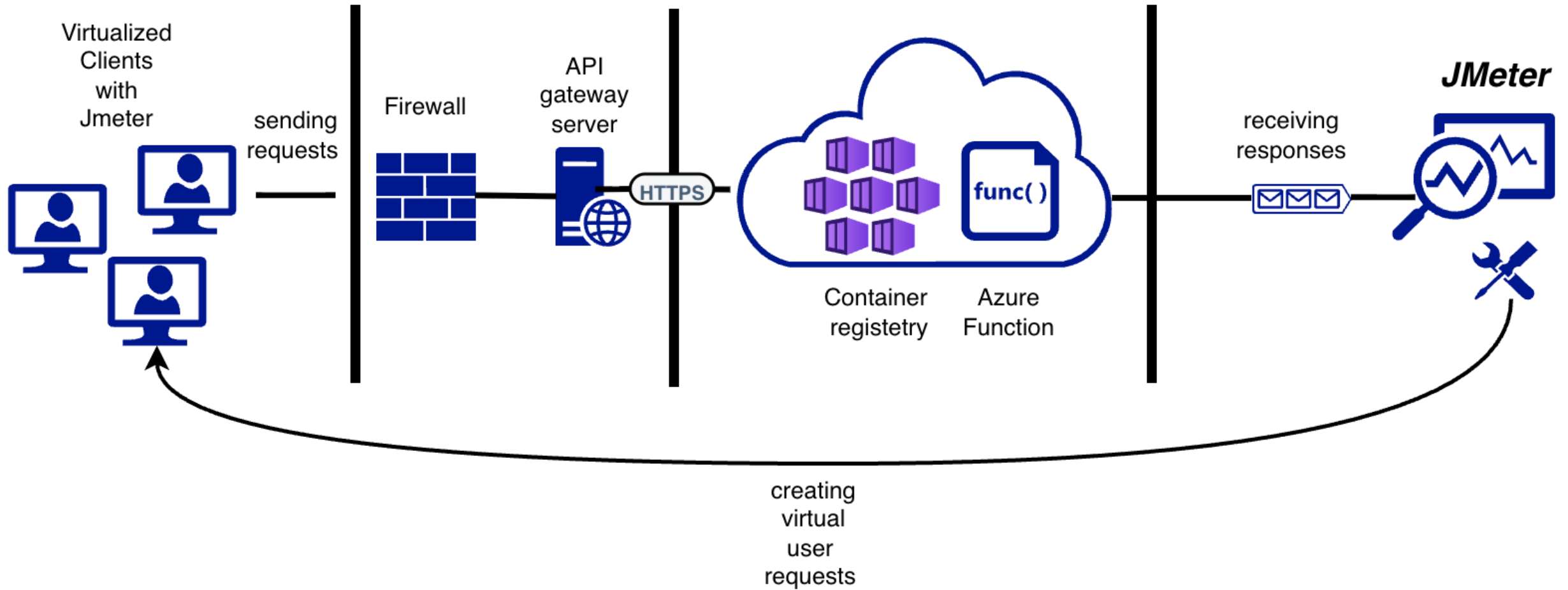
Resume Matching Application

Lightweight NLP using TF-IDF, cosine similarity, and skill extraction.

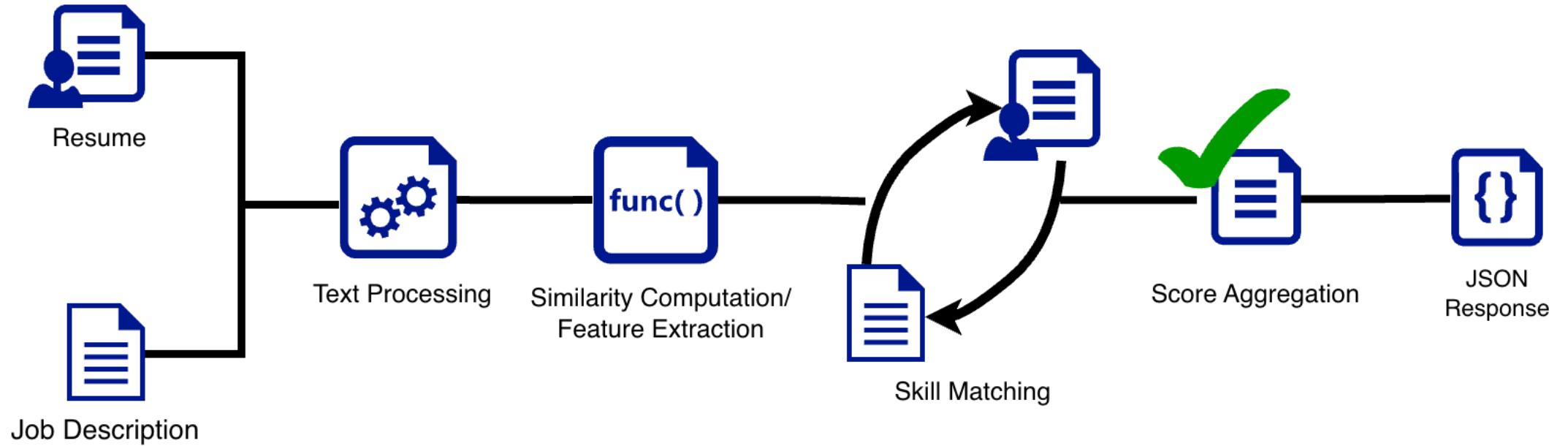
Performance Evaluation

Apache JMeter measures latency, throughput and etc.

SYSTEM ARCHITECTURE



FUNCTION METHODOLOGY



EVALUATION METRICS



COLD START



LOAD TESTS



STRESS TESTS



SPIKE TEST



PAYLOAD
TESTS



MIXED
WORKLOAD



PARETO
ANALYSIS

Abstract geometric lines in a light brown color, forming various polygons and intersecting lines on the left side of the page.

COLD START

TEST CONFIGURATION:

Thread Group

Name:

Comments:

Action to be taken after a Sampler error

☒ Continue ☐ Start Next Thread Loop ☐ Stop Thread ☐ Stop Test ☐ Stop Test Now

Thread Properties

Number of Threads (users):

Ramp-up period (seconds):

Loop Count: ☒ Infinite

☒ Same user on each iteration

☐ Delay Thread creation until needed

☒ Specify Thread lifetime

Duration (seconds):

Startup delay (seconds):

- Number of threads: 1
- Loop count : infinite
- Duration: 10s
- Think time: 2000ms (with poisson Random Timer)
- Https Trigger with 1KB payload

TEST RESULT – OVERALL STAT

Test and Report information

Source file	"cold_results.jtl"
Start Time	"7/18/26, 5:33 PM"
End Time	"7/18/26, 5:38 PM"
Filter for display	" "

APDEX (Application Performance Index)

Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
0.993	500 ms	1 sec 500 ms	Total
0.993	500 ms	1 sec 500 ms	HTTP Request

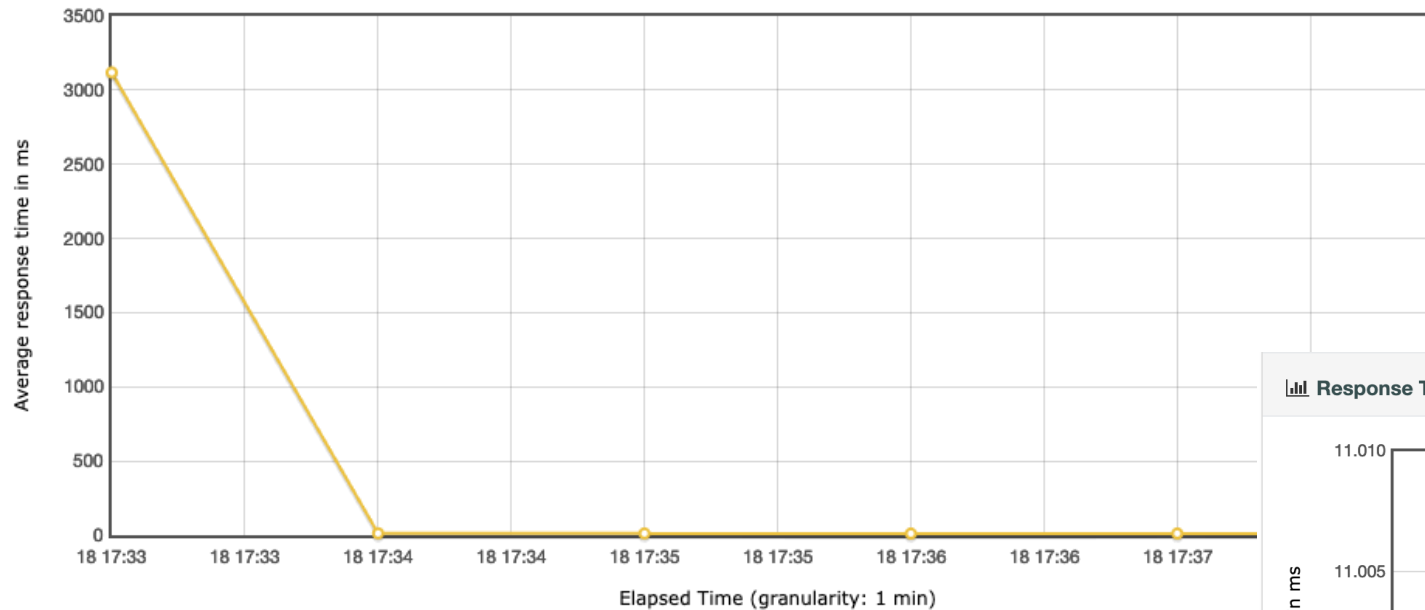
Requests Summary



Statistics

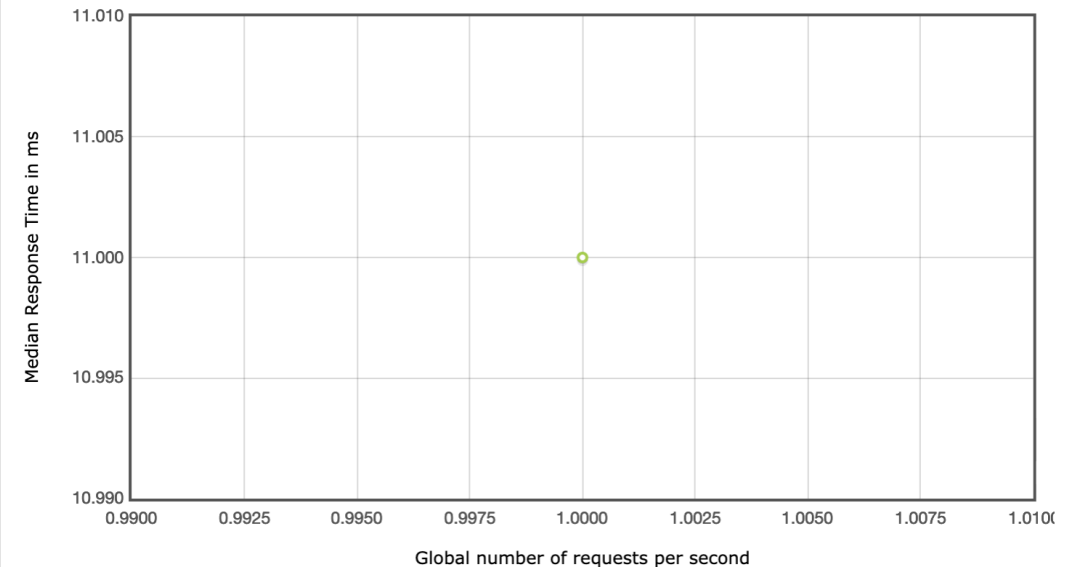
Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	134	0	0.00%	220.31	9	27973	11.00	13.50	18.00	18197.50	0.45	0.12	0.22
HTTP Request	134	0	0.00%	220.31	9	27973	11.00	13.50	18.00	18197.50	0.45	0.12	0.22

TEST RESULT – RESPONSE TIME



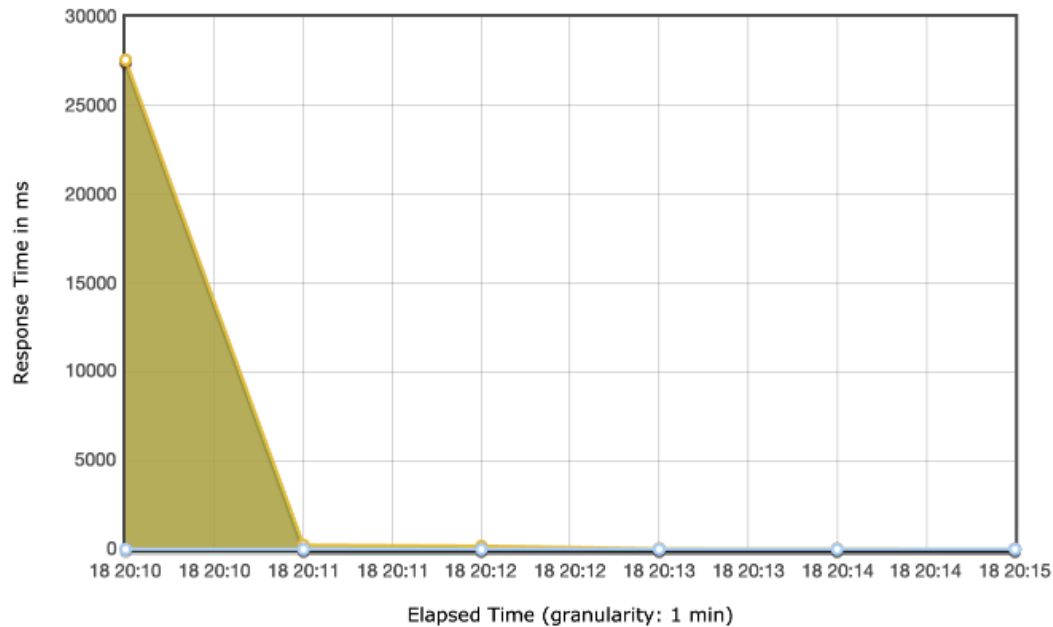
- the average response time spikes to **over 3,100 ms (3.1 seconds)**
- Immediately after, it drops off a cliff to a flat baseline of roughly **11 ms**

Response Time Vs Request

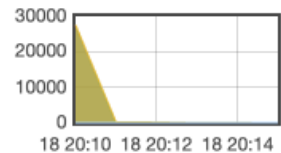


TEST RESULT- RESPONSE TIME WITH PERCENTILES

Response Time Percentiles Over Time (successful responses)



Zoom :

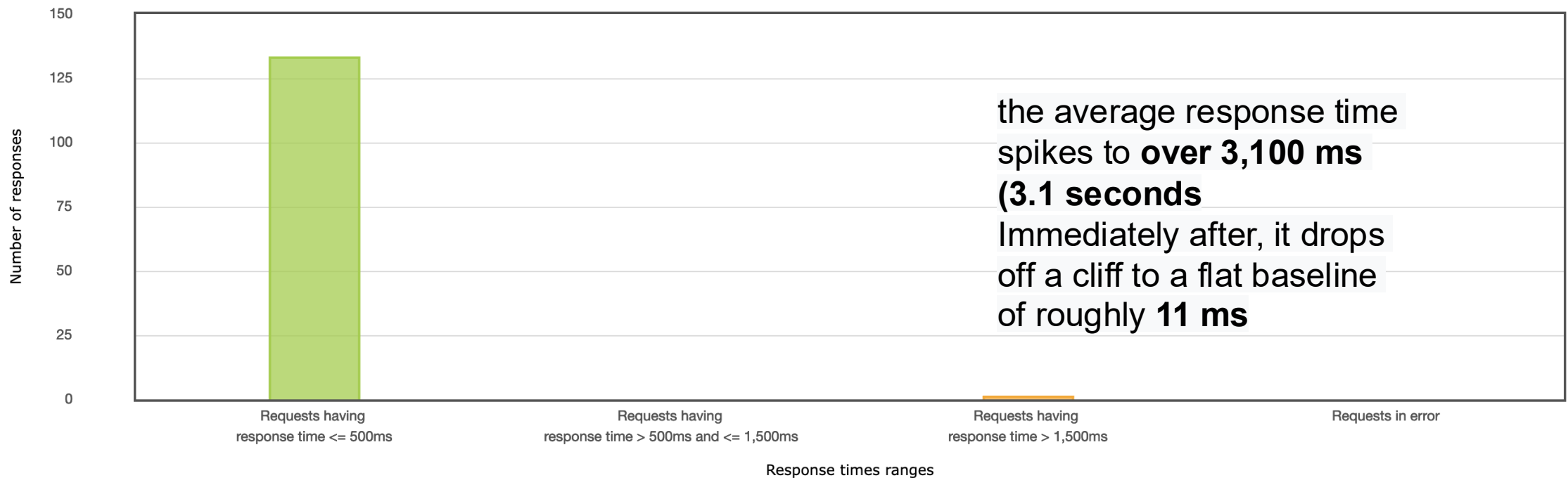


90th percentile 95th percentile 99th percentile Max Median Min

- The **Max** response time line reaches a staggering **28,000 ms (28 seconds)** at the very beginning, while the median remains near zero.
- **The Calculation of delta between cold start and warm start**
- To isolate the provisioning penalty, we use a simple subtraction:
 - $\Delta = T_{cold} - T_{warm}$
- **Cold Start Time: 28,000 ms**
- **Warm Start Time: 11 ms**
- **$\Delta = 28000 - 11 = 27989 \text{ ms}$**

TEST RESULT- RESPONSE TIME OVERVIEW

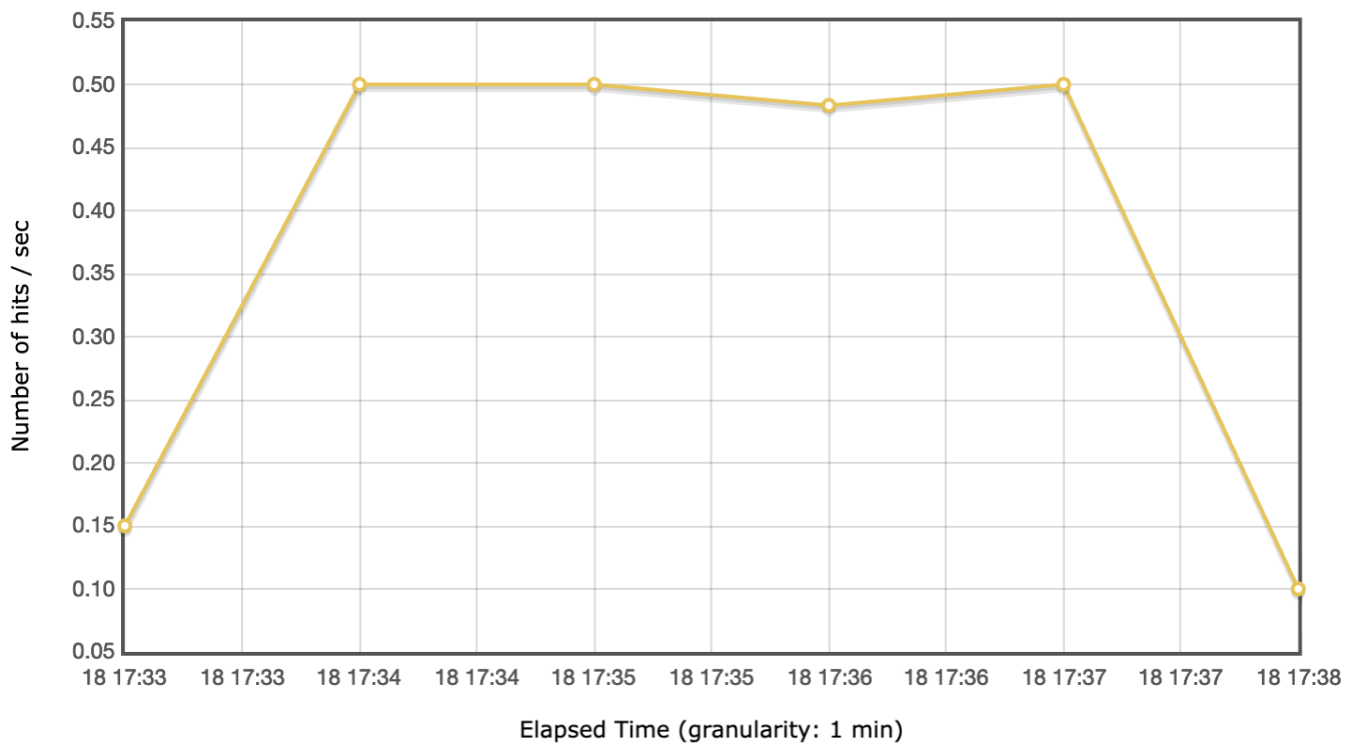
 Response Time Overview



 Requests having response time <= 500ms  Requests having response time > 1,500ms  Requests having response time > 500ms and <= 1,500ms  Requests in error

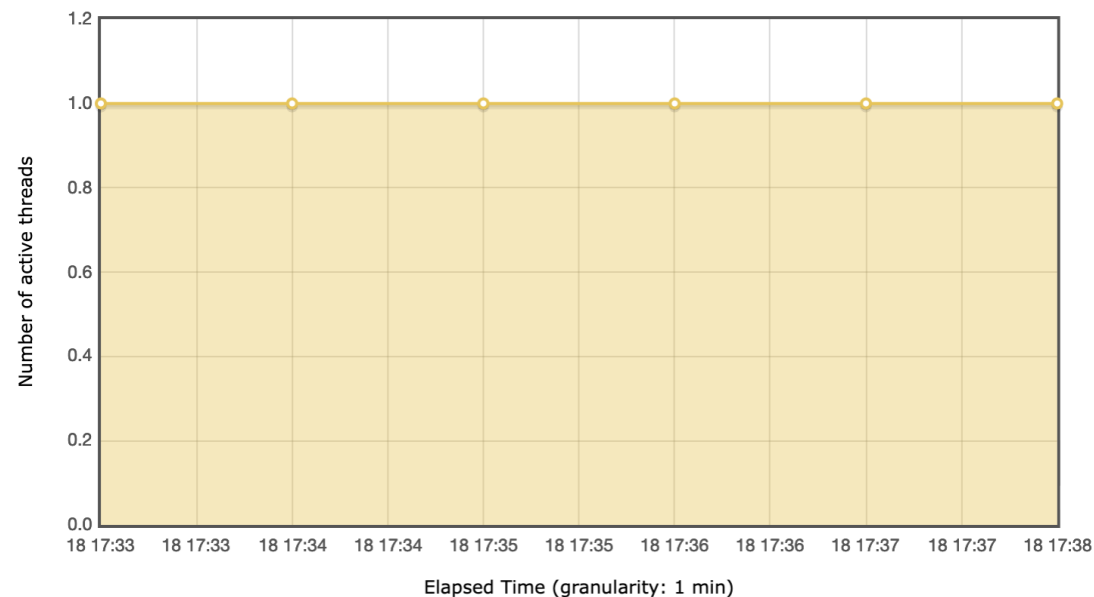
TEST RESULT- THROUGHPUT

 Hits Per Second



Active Threads is perfectly flat at **1.0**, and Hits Per Second plateaus exactly at **0.5 hits/sec**.

 Active Threads Over Time



Abstract geometric lines in a light brown color, forming various polygons and intersecting lines on the left side of the slide.

LOAD TEST

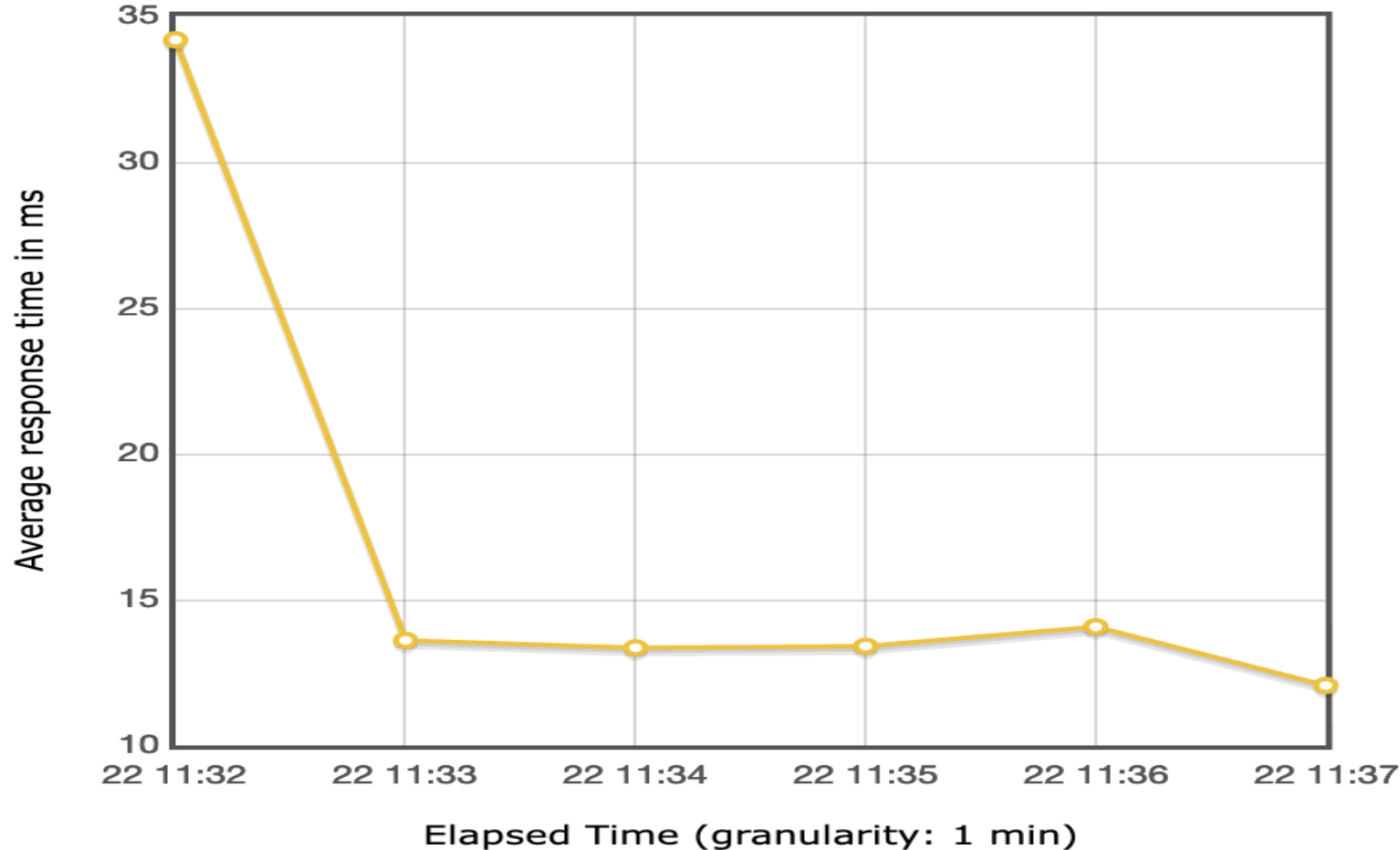
TEST CONFIGURATION:

- **Number of threads:** 50
- **Loop count:** Infinite
- **Duration:** 300 s
- **Ramp-up period:** 10 s
- **Think time:** 2000 ms (Poisson Random Timer)
- **HTTP Trigger:** HTTPS POST request with approximately **1 KB** JSON payload

COMPARISON TABLE

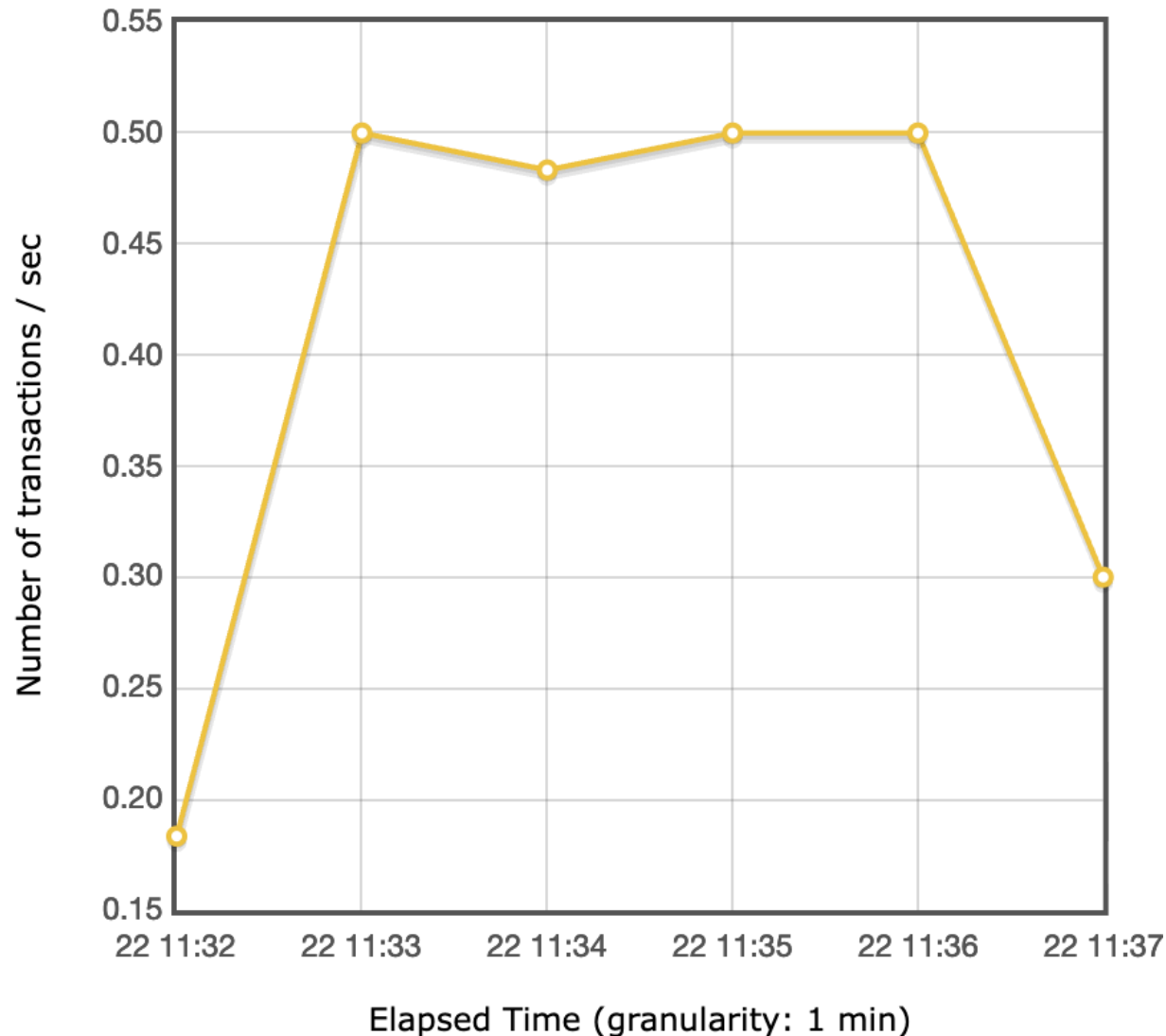
Concurrent Users	Average Response Time (ms)	95th Percentile (ms)	Throughput (TPS)	Error Rate (%)	APDEX
10	169.09	25.35	0.46	0.00	0.993
25	15.07	18.55	0.50	0.00	1.000
50	14.95	16.00	0.50	0.00	1.000

RESPONSE TIMES OVER TIME (50 USERS)



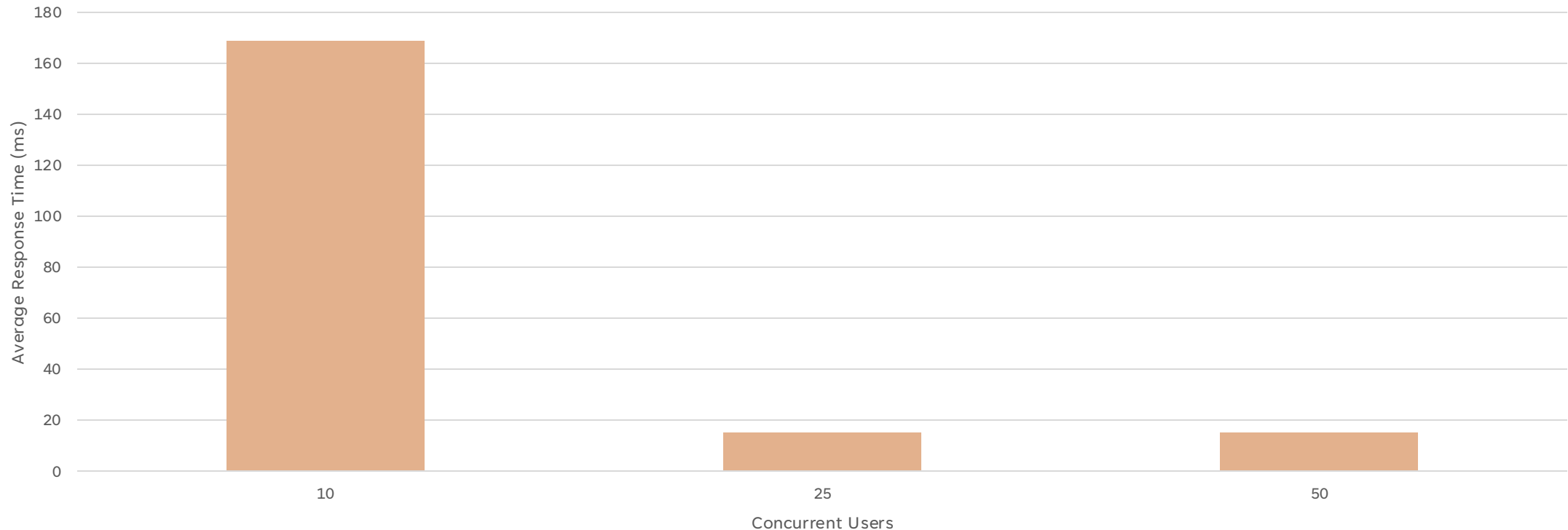
The response time decreased sharply from approximately **34 ms to 13 ms** after the initial minute and then remained stable between **12 ms and 14 ms** throughout the remainder of the **300-second load test**. This demonstrates that the Azure Function maintained low latency and consistent performance while handling **50 concurrent users**.

TRANSACTIONS PER SECOND (50 USERS)



The throughput increased rapidly from approximately **0.18 TPS** to a stable rate of **0.50 TPS** after the initial minute. It remained consistently between **0.48 and 0.50 TPS** throughout the load test, indicating that the Azure Function sustained a steady request processing rate under **50 concurrent users**. The final decrease to **0.30 TPS** occurred as the test concluded and is expected behavior.

IMPACT OF CONCURRENT USERS ON AVERAGE RESPONSE TIME

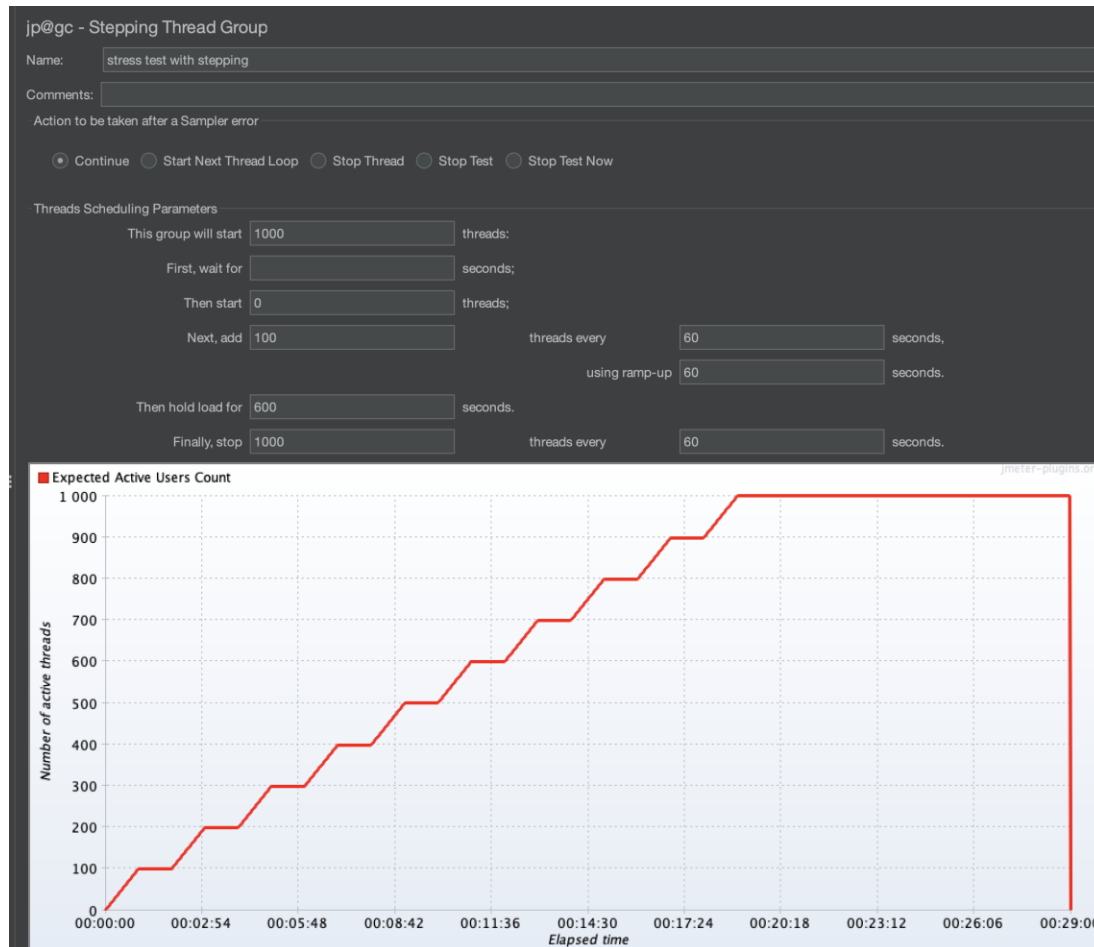


- The average response time was **highest at 10 concurrent users (169.09 ms)**.
- Response time **decreased significantly** when the load increased from **10 to 25 concurrent users**.
- The average response time remained **stable between 25 and 50 concurrent users** (15.07 ms and 14.95 ms, respectively).
- The results indicate that the Azure Function maintained **consistent performance** under the tested load conditions after the initial execution.

Abstract geometric lines in a light brown color, forming various polygons and intersecting lines on the left side of the page.

STRESS TEST

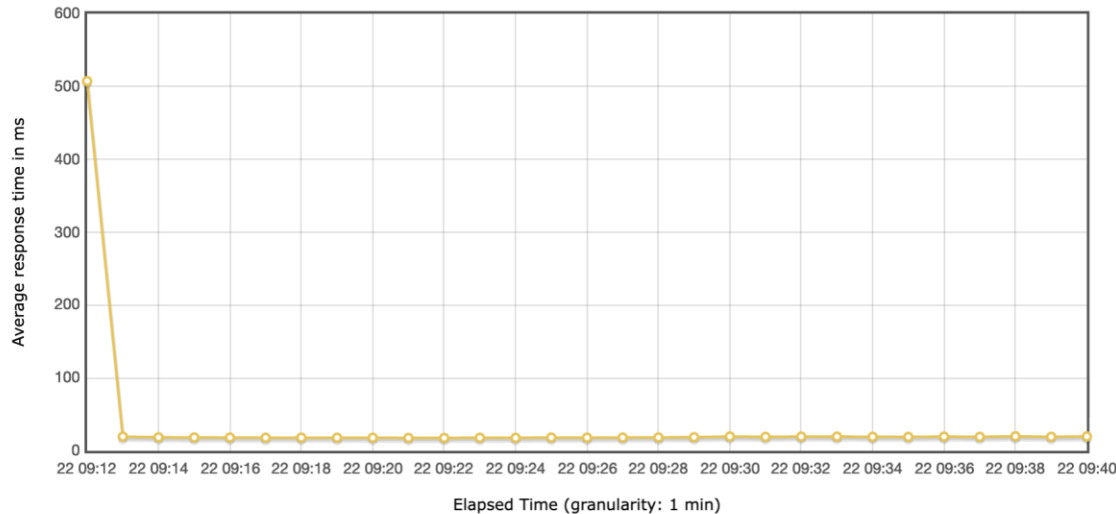
TEST CONFIGURATION 1: CONSTANT PAYLOAD AND 1000 USERS



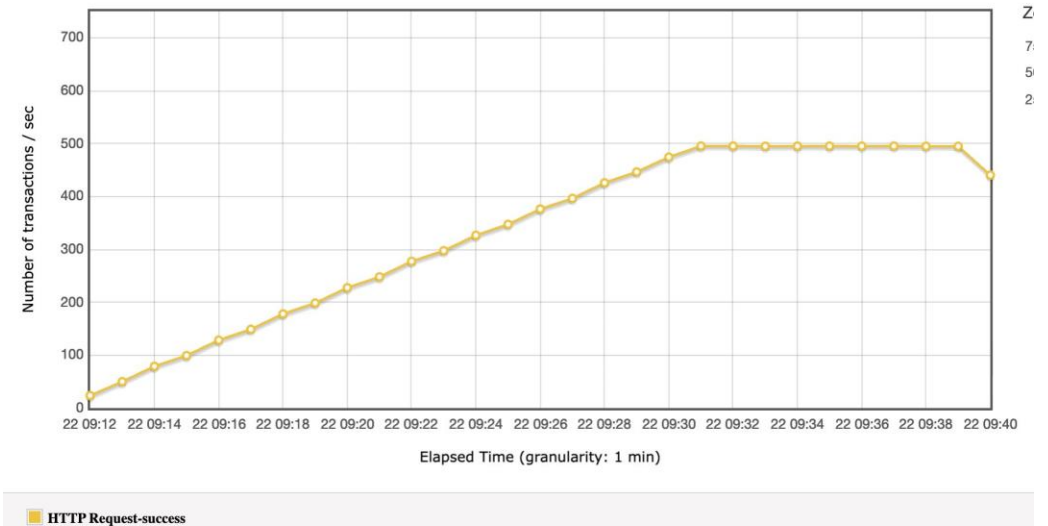
- Payload was kept constant
- Keeping the think time 2000ms
- Giving a 60s ramp up
- Adding 100 users by every 60 s
- Holding period 10 mins

RESPONSE TIME AND THROUGHPUT

Response Times Over Time



Transactions Per Second



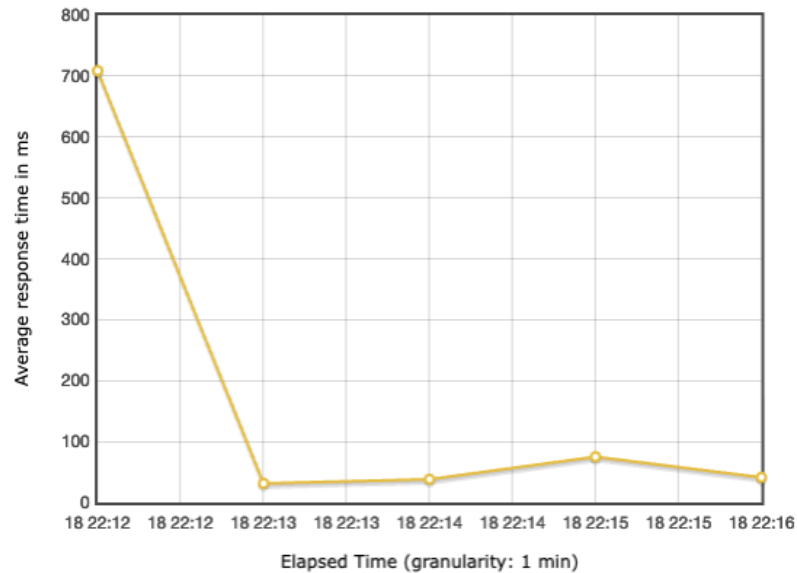
- The Response Times chart visually isolates the initial cold-start penalty, a distinct average latency spike at the test's onset as Azure provisions the underlying compute resources—before instantly dropping into a perfectly flat, highly optimized steady-state for the remaining duration.
- After the initial measurement, **response times consistently stayed around 16 - 17 ms.**
- 2nd chart demonstrates a linear load ramp-up that eventually hits a strict horizontal ceiling at approximately 500 requests per second, mathematically identifying the maximum processing capacity of the FaaS architecture before queue accumulation begins.

TEST CONFIGURATION 2: MASSIVE CROWD METHOD WITH 1000 USERS

- **Number of threads:** 1000
- **Loop count:** Infinite
- **Duration:** 300 s
- **Ramp-up period:** 100 s
- **Think time:** 2000 ms (exponential increment)
- **HTTP Trigger:** HTTPS POST request with approximately **1 KB** JSON payload

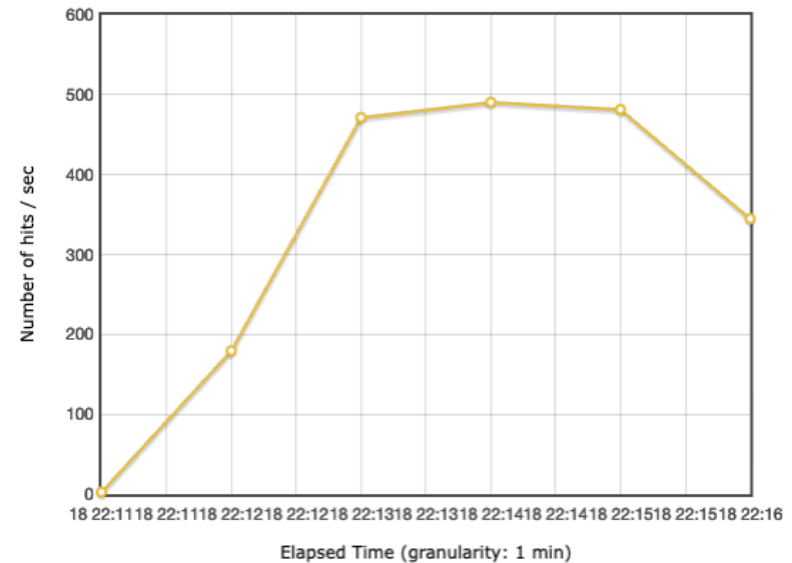
RESPONSE TIME AND THROUGHPUT

Response Times Over Time

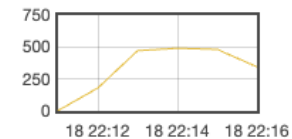


- The 36.7-second maximum latency outlier directly isolates the infrastructure cold-start penalty.
- It validates that once resources are allocated, the closed queuing network handles high-concurrency bursts efficiently.
- After the initial measurement, **response times consistently stayed around 14 ms.**

Hits Per Second



Zoom :

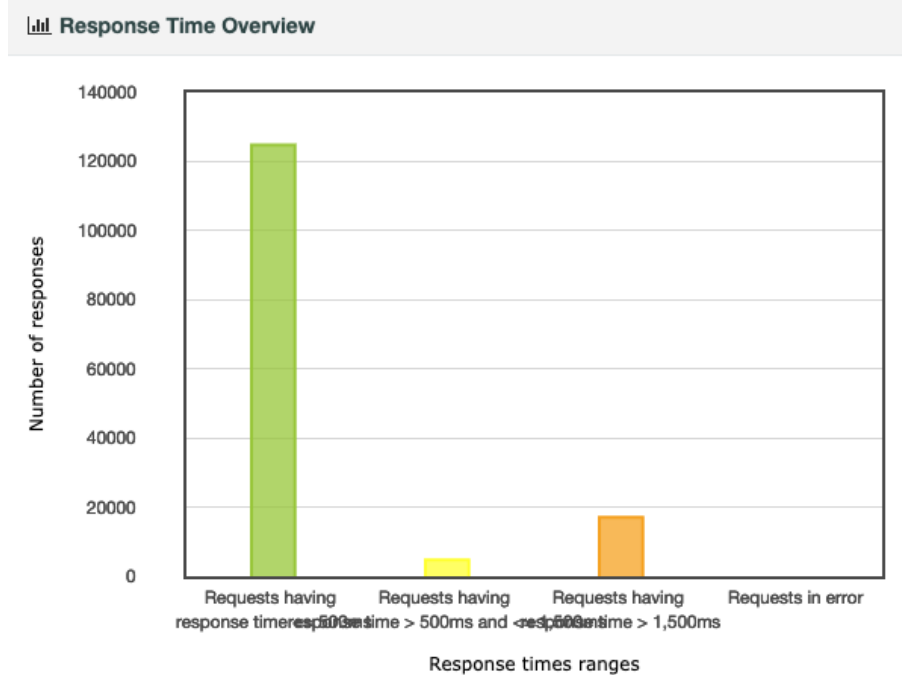
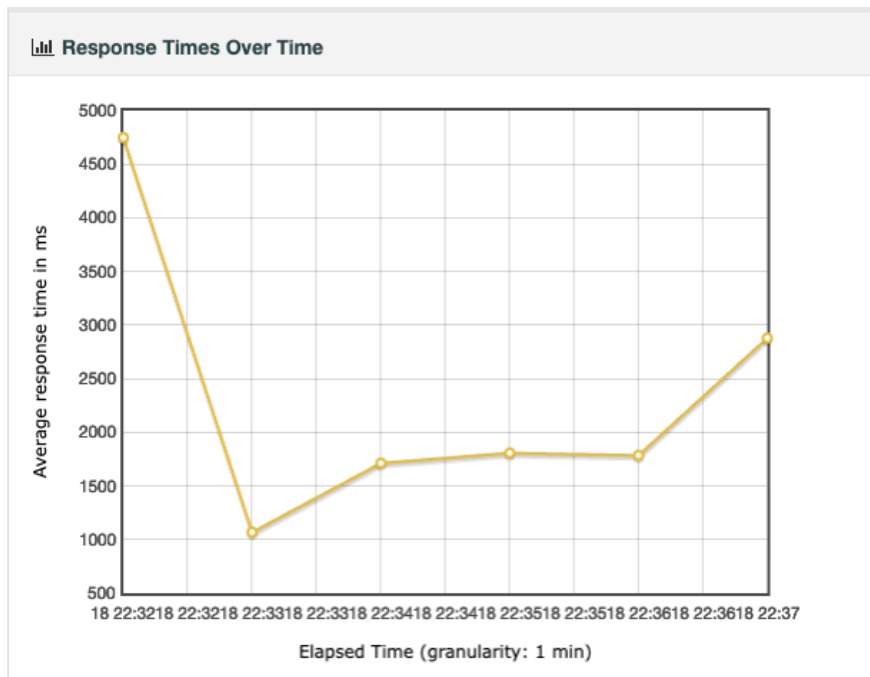


hitsPerSecond

TEST CONFIGURATION 3: QUEUE FLOOD METHOD WITH 1000 USERS

- Number of threads: 1000
- Loop count: Infinite
- Duration: 300 s
- Ramp-up period: 100 s
- Think time **is made to zero**
- HTTP Trigger: HTTPS POST request with approximately **100 KB JSON** payload

RESPONSE TIME VARIATIONS



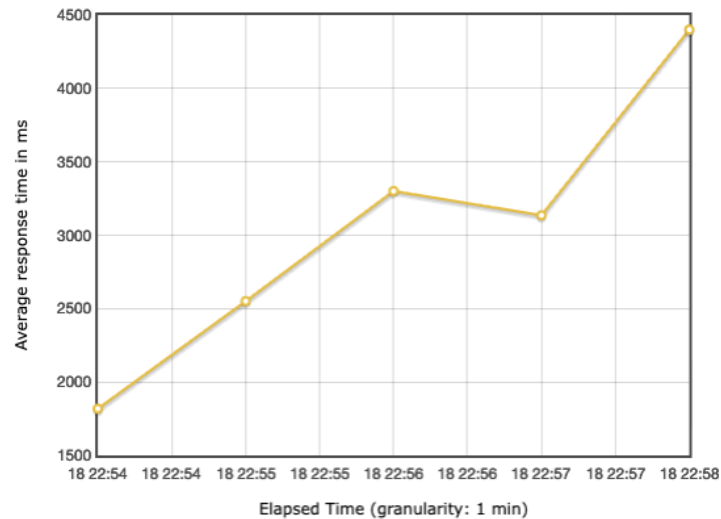
- After the cold start, even the response times get much lower, it shows that, again it is climbing without a clear pattern.
- And the good thing here is that most requests are responded in below 500ms.

TEST CONFIGURATION 4: 2000 USERS WITH A SMALL THINK TIME

- **Number of threads:** 2000
- **Loop count:** Infinite
- **Duration:** 300 s
- **Think time:** 500ms
- **Ramp-up period:** 100 s
- **HTTP Trigger:** HTTPS POST request with approximately 1KB JSON payload

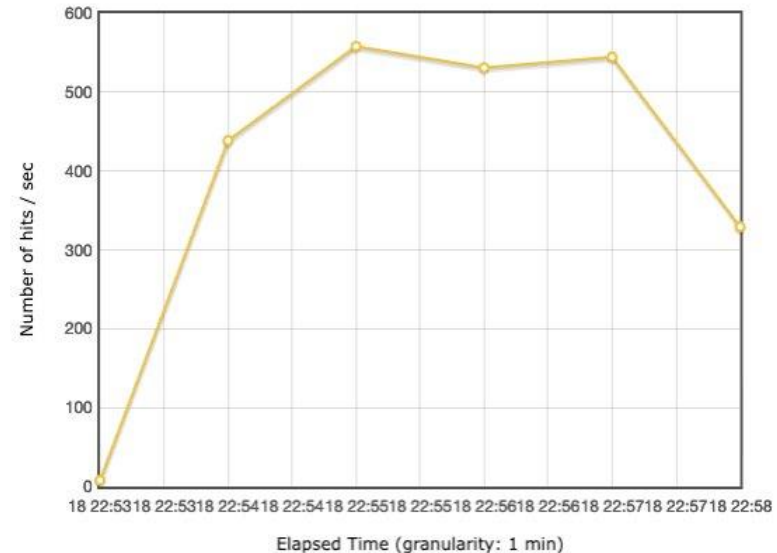
RESPONSE TIME AND THROUGHPUT

Response Times Over Time



The average response time continuously climbs from **~1,800 ms to over 4,400 ms**. This proves that the incoming traffic is arriving faster than the server can process it, causing a **steadily growing queue delay**.

Hits Per Second



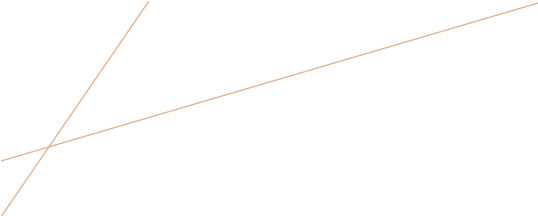
The throughput ramps up but hits a hard, flat plateau at roughly **550 requests per second**. This flatline mathematically identifies the absolute maximum processing capacity of the load balancer.

COMPARISON OF DIFFERENT CONFIGURATIONS

Performance Metric	Massive Crowd	Queue Flood	2000 Users
Total Samples Processed	118,256	146,614	144,579
Error Rate (%)	0.00% (0 Fails)	0.00% (0 Fails)	0.00% (0 Fails)
Throughput (X)	396.43 req/s	483.53 req/s	474.03 req/s
Median Latency (50th pct)	14.00 ms	20.00 ms	41.00 ms
90th Percentile Latency	77.00 ms	17,872.40 ms (~17.8 s)	27,483.90 ms (~27.5 s)
95th Percentile Latency	136.95 ms	19,264.00 ms (~19.3 s)	30,054.95 ms (~30.0 s)
99th Percentile Latency	464.00 ms	19,584.99 ms (~19.6 s)	31,303.99 ms (~31.3 s)
APDEX Score (T=500ms)	0.986 (Near Perfect)	0.867 (Satisfactory)	0.834 (Degraded)

Abstract geometric lines in a light beige color, forming various polygons and intersecting lines on the left side of the slide.

SPIKE TEST



TEST CONFIGURATION:

- **Number of Threads (Users):** 50
- **Ramp-up Period:** 0 seconds
- **Loop Count:** Infinite
- **Test Duration:** 300 seconds
- **Think Time:** 2000 ms (*Poisson Random Timer*)
- **HTTP Trigger:** ~1 KB

RESPONSE TIMES OVER TIME

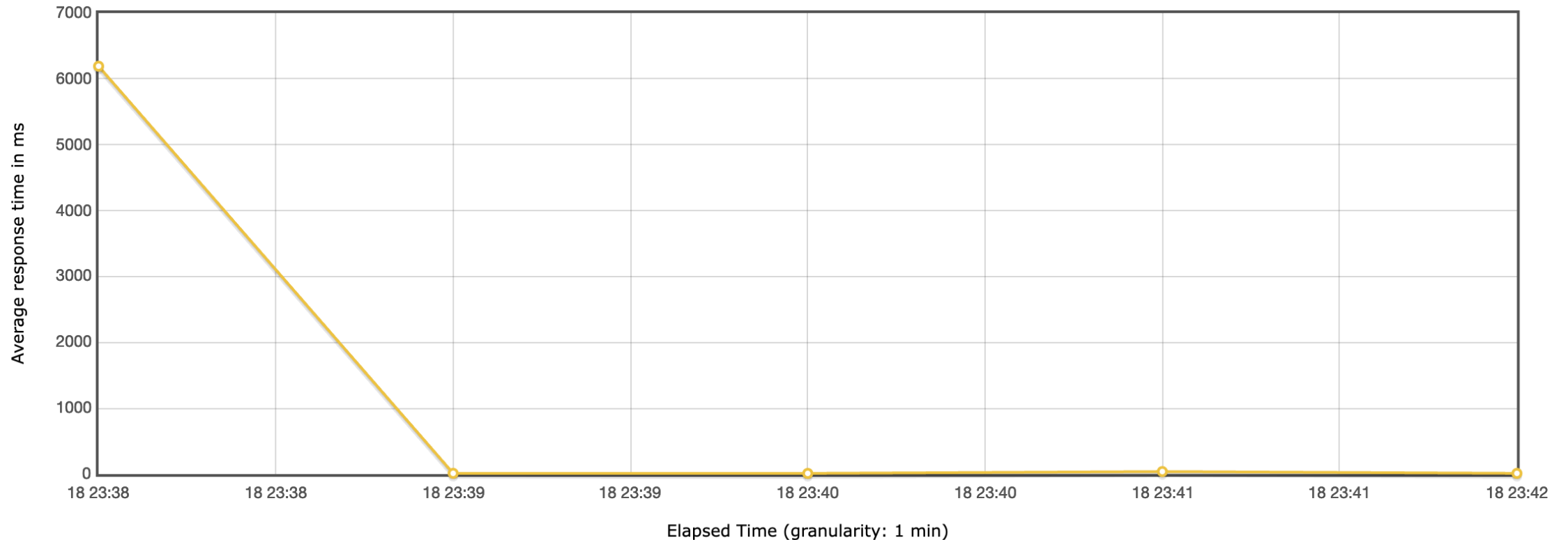


Figure: Response time variation during the spike test.

The average response time initially peaked at approximately **6,200 ms** due to the sudden traffic spike generated with a **0-second ramp-up**. However, response time rapidly decreased to **nearly 0 ms** within the following minute and remained stable for the rest of the test. This demonstrates that the Azure Function recovered quickly and maintained reliable performance under the spike workload **without request failures**.

TRANSACTIONS PER SECOND

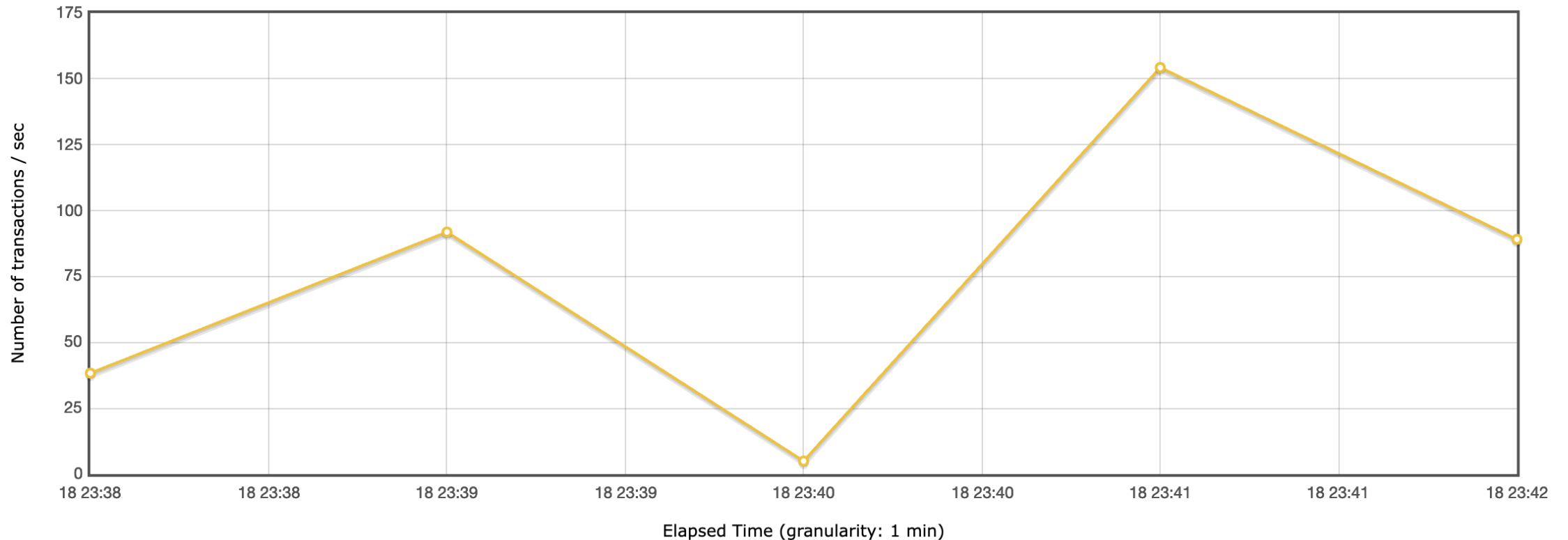


Figure: Response time variation during the spike test.

The throughput initially increased from approximately **38 TPS to 92 TPS**, briefly dropped to 5 TPS during the immediate spike response, and then reached a peak of approximately **154 TPS** as the Azure Function scaled to handle the workload. It finally stabilized at around **89 TPS**, demonstrating successful request processing and recovery **without any request failures**.

ACTIVE THREADS OVER TIME

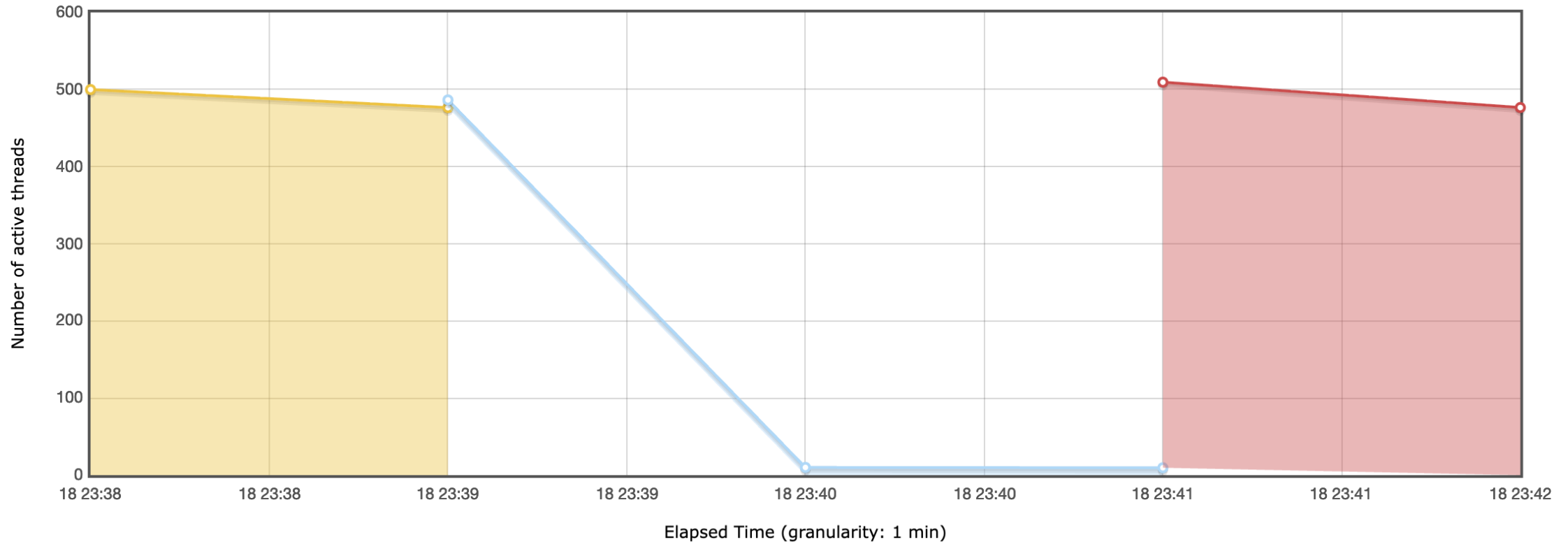
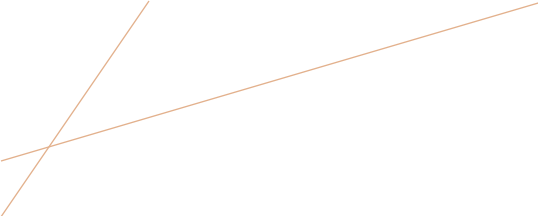


Figure: Active virtual users during the spike test.

All virtual users were initiated simultaneously using a **0-second ramp-up**, creating an immediate spike workload of approximately **500 active threads**. The active thread count remained consistently high throughout the test and decreased only as the test approached completion, confirming **sustained concurrent user activity** and successful execution of the **spike workload**.

A series of thin, light brown lines forming an abstract, overlapping geometric pattern on the left side of the slide. The lines intersect to create various polygonal shapes, some of which are filled with a very light beige color.

PAYLOAD SIZE TEST



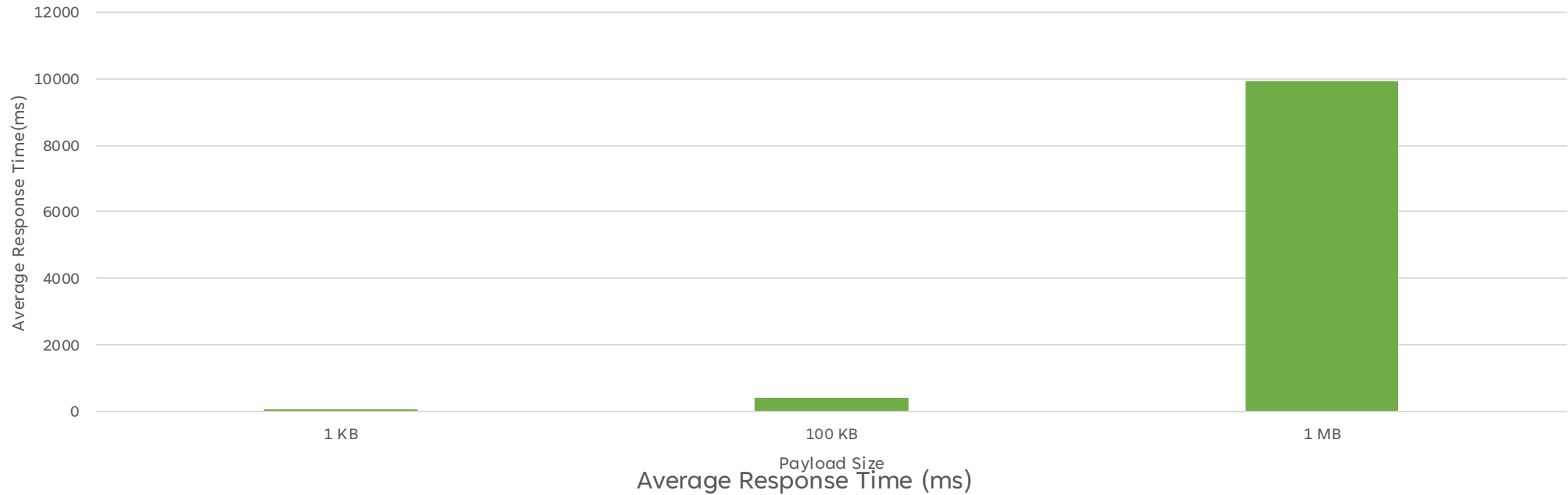
TEST CONFIGURATION:

- **Number of Threads (Users):** 50
- **Ramp-up Period:** 0 seconds
- **Loop Count:** Infinite
- **Test Duration:** 300 seconds
- **Think Time:** 2000 ms (*Poisson Random Timer*)
- **Payload Sizes:** 1 KB, 100 KB, and 1 MB

PERFORMANCE COMPARISON

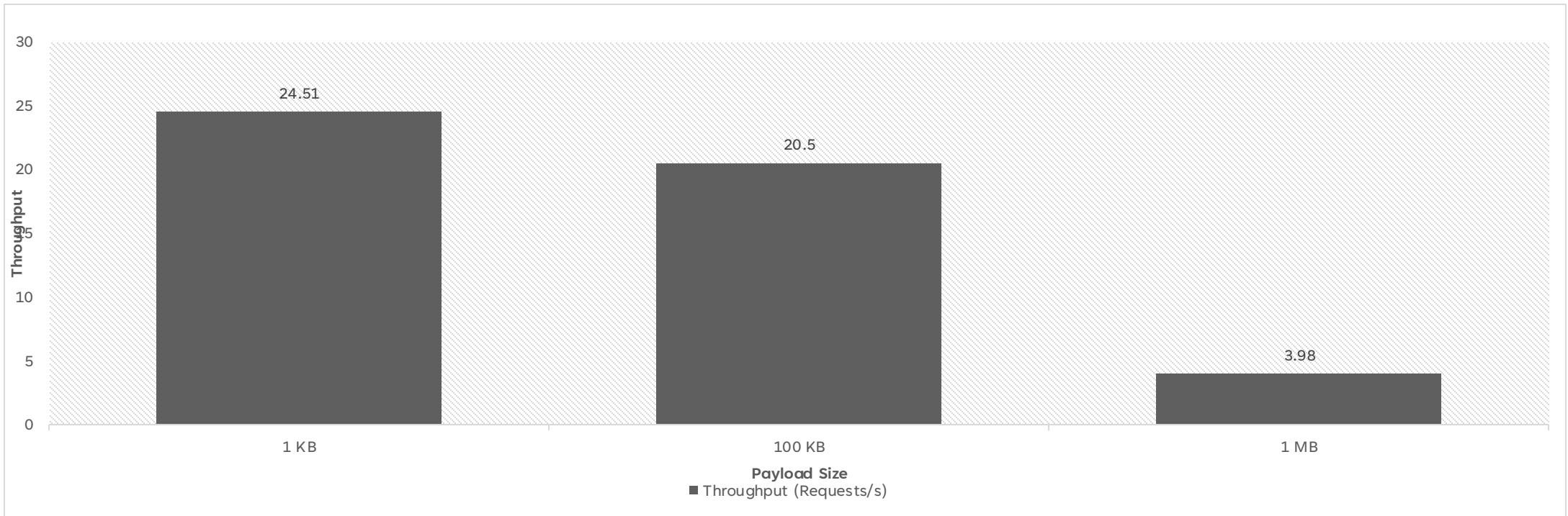
Performance Metric	1 KB Payload	100 KB Payload	1 MB Payload	Performance Trend / Observation
Average Response Time	13.47 ms	407.87 ms	9,913.71 ms	Response time increased significantly with larger payload sizes.
Throughput	24.51 req/s	20.50 req/s	3.98 req/s	Throughput decreased substantially as payload size increased.
95th Percentile Response Time	19.00 ms	837.35 ms	27,901.50 ms	Tail latency increased dramatically, indicating slower request completion for larger payloads.
Error Rate	0.00%	0.00%	0.00%	Stable operation with no request failures across all tests.
APDEX Score	1	0.88	0	User satisfaction declined significantly as payload size increased.

IMPACT OF PAYLOAD SIZE ON AVERAGE RESPONSE TIME



- The average response time **increased substantially** as the payload size increased from **1 KB to 1 MB**.
- The **1 KB** payload achieved the fastest response time (**13.47 ms**), while the **1 MB** payload recorded the highest response time (**9913.71 ms**).
- These results indicate that **larger payloads significantly increase processing time**, leading to higher latency in Azure Functions.

IMPACT OF PAYLOAD SIZE ON THROUGHPUT



- Throughput **decreased as the payload size increased**, indicating reduced request processing capacity.
- The **1 KB** payload achieved the highest throughput (**24.51 requests/s**), whereas the **1 MB** payload achieved only **3.98 requests/s**.
- The results demonstrate that **larger payloads reduce the number of requests the Azure Function can process per second**, highlighting the impact of payload size on system scalability.

BEST-CASE VS WORST-CASE RESPONSE TIME

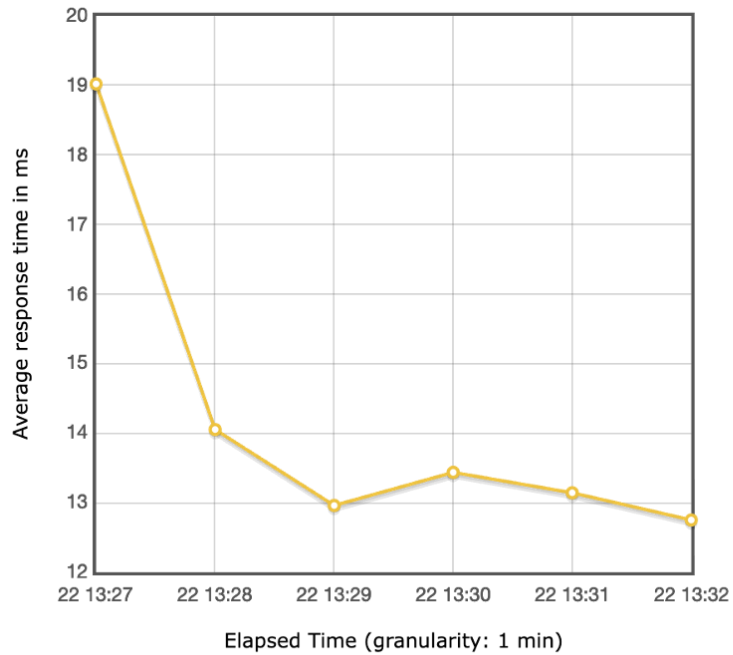


Figure: Response time variation during the Payload Size Testing of 1KB

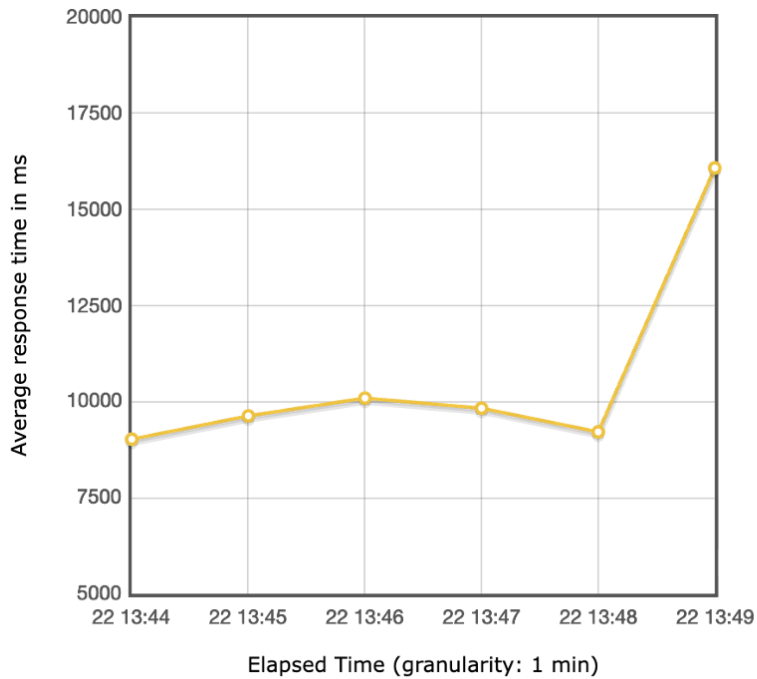


Figure: Response time variation during the Payload Size Testing of 1MB

For the **1 KB payload**, the average response time decreased from approximately **19 ms** and remained stable between **13–14 ms**, demonstrating consistently low latency under a small payload. In contrast, the **1 MB payload** maintained response times around **9,000–10,000 ms** before increasing sharply to approximately **16,000 ms**, highlighting the significant performance impact of processing larger payloads.

BEST-CASE VS WORST-CASE TRANSACTIONS PER SECOND

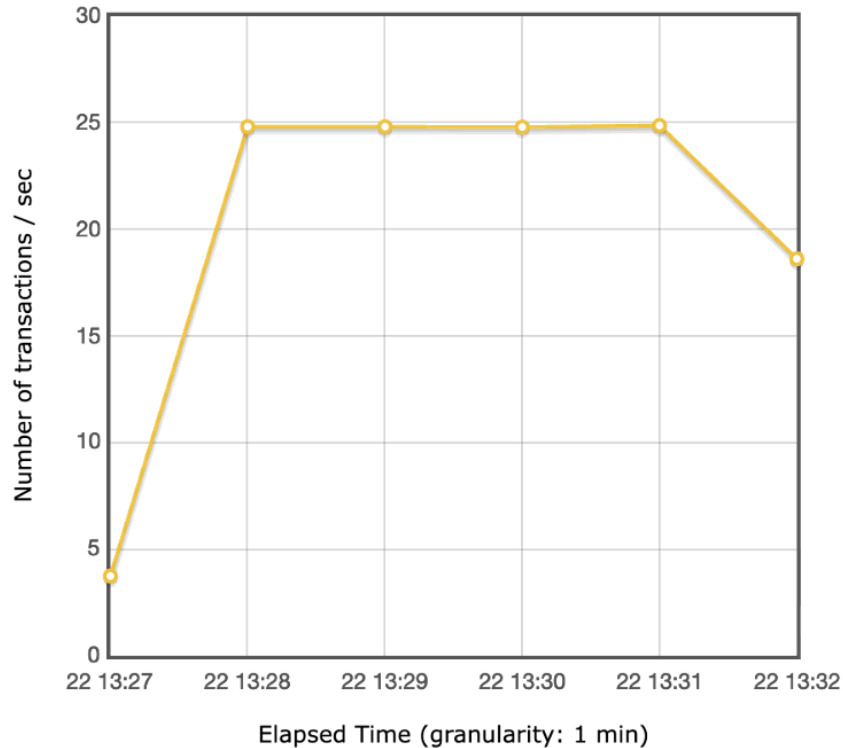


Figure: Throughput during the Payload Size Testing of 1KB

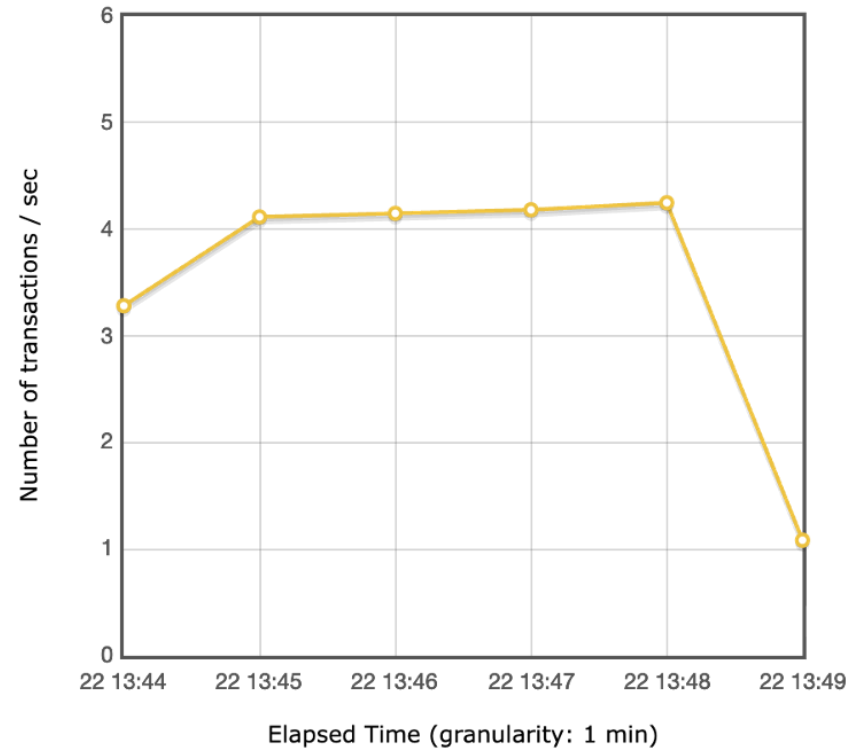


Figure: Throughput during the Payload Size Testing of 1MB

For the **1 KB payload**, throughput increased rapidly from approximately **4 TPS** to a stable rate of **25 TPS**, demonstrating efficient request processing. In contrast, the **1 MB payload** achieved only **4 TPS** at its peak and declined to approximately **1 TPS** by the end of the test, indicating that larger payloads significantly reduced the application's processing capacity.

A series of thin, light brown lines on the left side of the slide, forming an abstract, overlapping geometric pattern that resembles a stylized, multi-faceted shape or a series of intersecting planes.

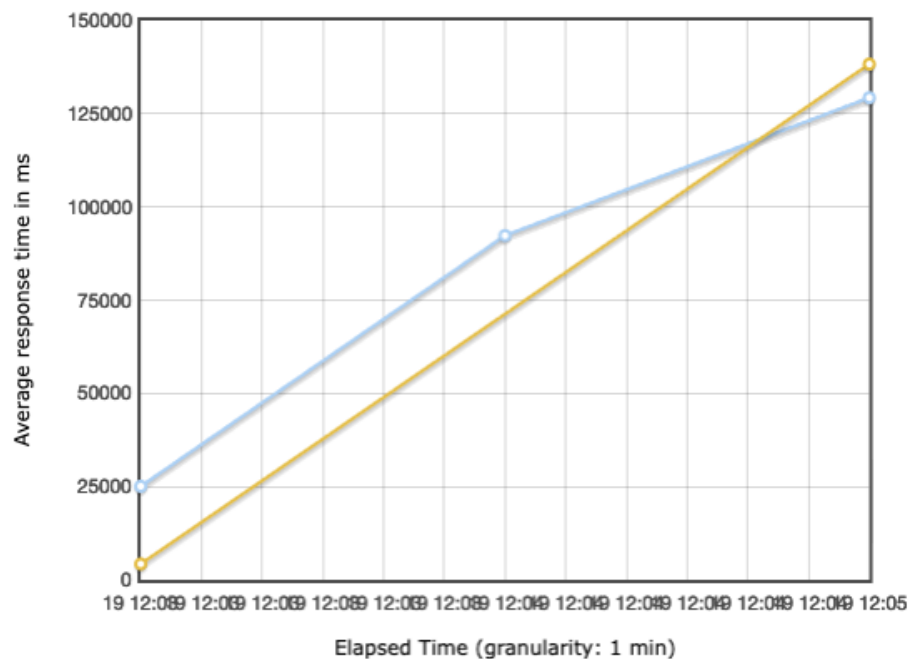
MIXED WORKLOAD

TEST CONFIGURATION:

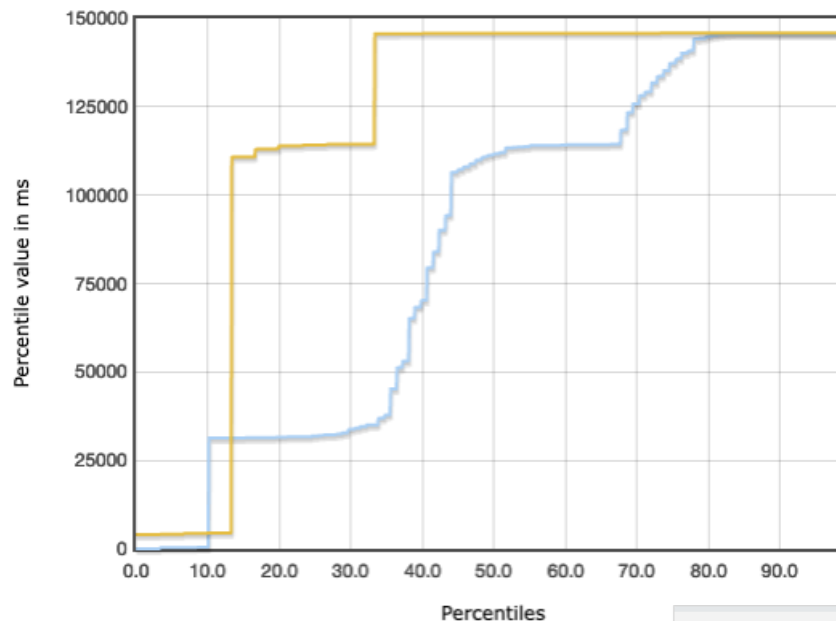
- **Number of Threads (Users):** 100
- **Ramp-up Period:** 0 seconds
- **Loop Count:** Infinite
- **Test Duration:** 60 seconds
- **Think Time:** 2000 ms
- **HTTP Trigger 1:** 1 KB With 80% throughput
- **HTTP Trigger 2:** 1 MB With 20% throughput

RESPONSE TIME VARIATIONS

Response Times Over Time



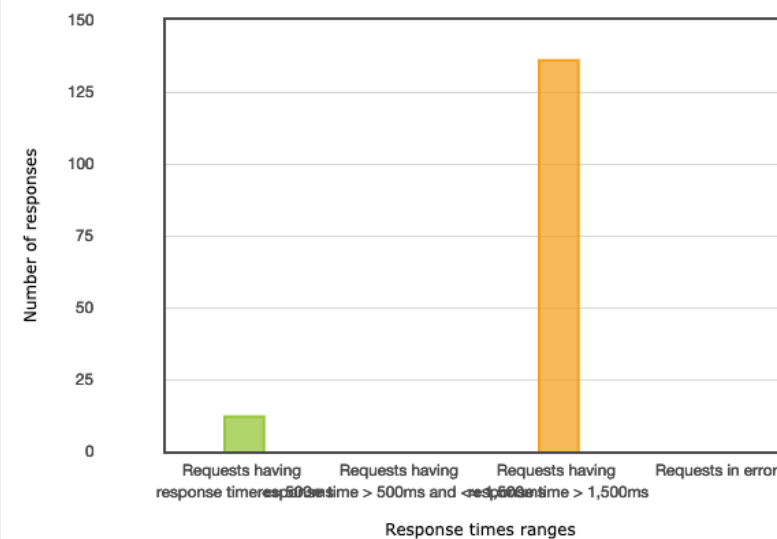
Response Time Percentiles



- Both 1 KB and 1 MB response times climb steadily over time, reaching **>125 seconds**.
- The 20% heavy payloads (1 MB) choke the shared ingress pipeline, forcing the 80% lightweight requests (1 KB) to wait in line.
- The percentile graph confirms that lightweight 1 KB requests suffer severe degradation, spiking drastically after the **40th percentile**.

- Blue -1KB
- Yellow -1MB

Response Time Overview



Abstract geometric lines in a light beige color, forming various polygons and intersecting patterns on the left side of the slide.

PARETO ANALYSIS

TEST CONFIGURATION:

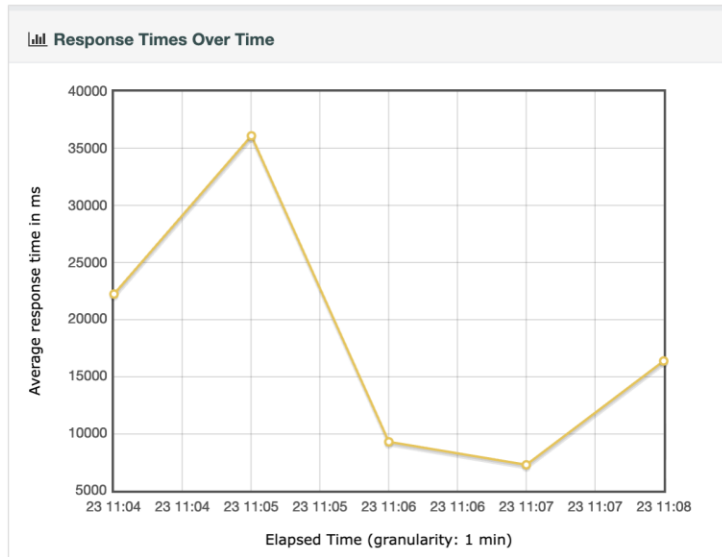
JMETER CONFIGURATION

- **Number of Threads (Users):** 50
- **Ramp-up Period:** 0 seconds
- **Loop Count:** Infinite
- **Test Duration:** 300 seconds
- **Think Time:** 2000 ms
- **HTTP Trigger:** ~1 KB

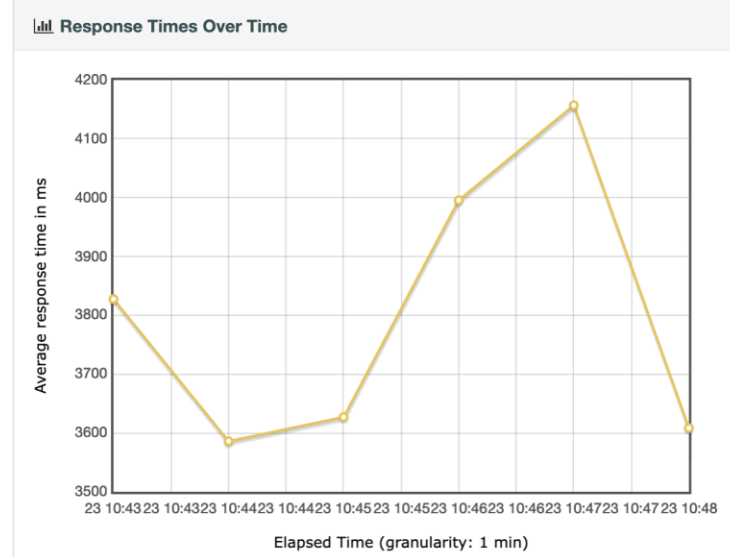
AZURE FUNCTION CONFIGURATION

- TEST 1 – 0.25 CPU, 0.5GB MEMORY
- TEST 2 – 0.5 CPU, 1.0GB MEMORY
- TEST 3 – 1 CPU, 2.0GB MEMORY

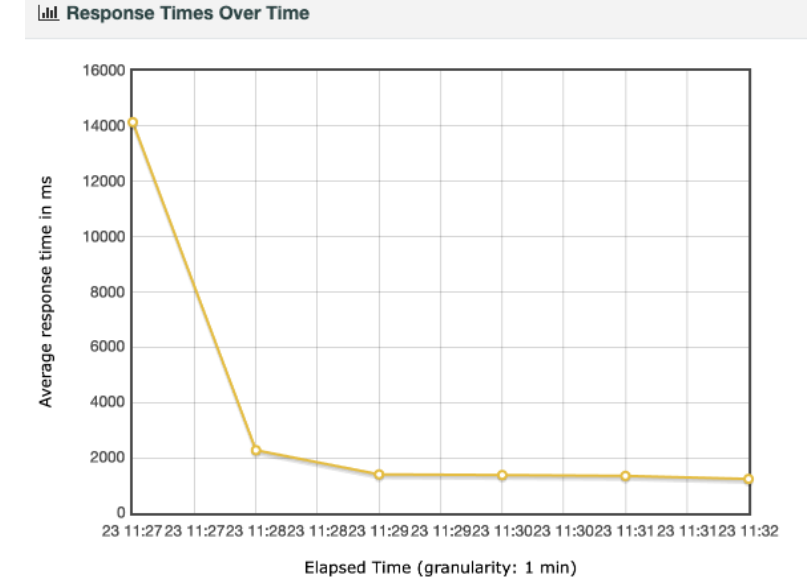
RESPONSE TIMES FOR DIFFERENT CONFIGURATION



0,5 GB
CONFIGURATION



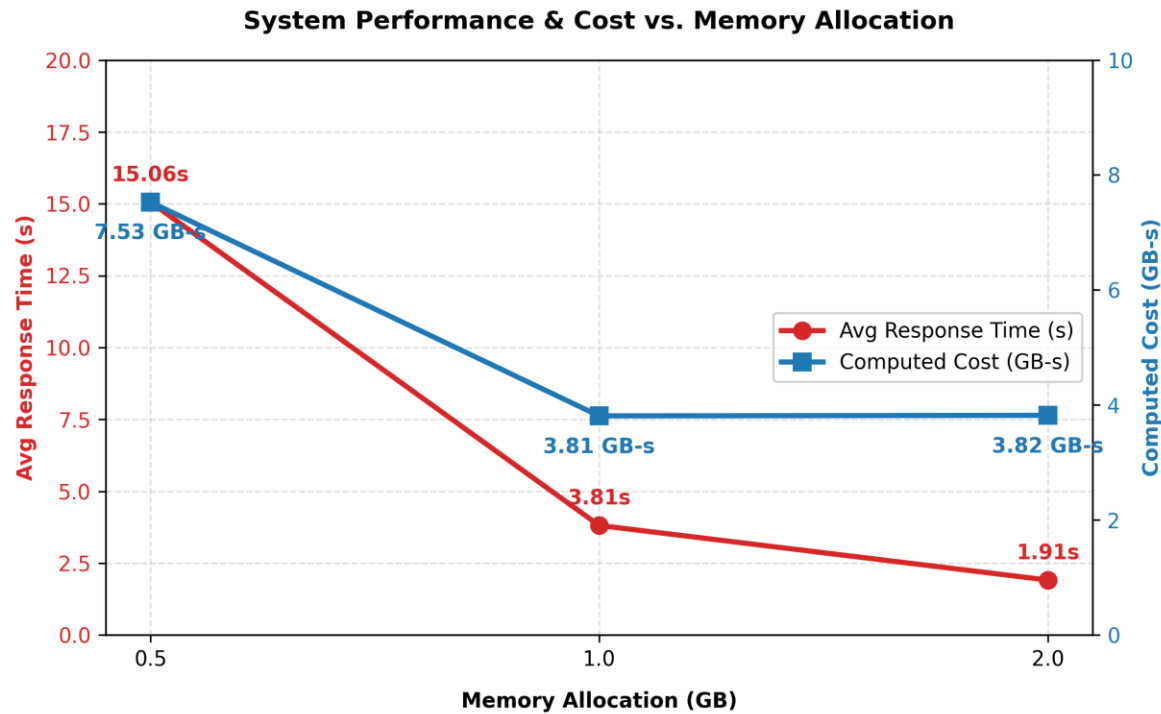
1GB
CONFIGURATION



2 GB
CONFIGURATION

THE COST-PERFORMANCE PARETO ANALYSIS

Memory Allocation (GB) [X-Axis]	Avg Response Time (s) [Left Y-Axis]	Computed Cost (GB-s) [Right Y-Axis]
0.5	15.06	7.53
1.0	3.81	3.81
2.0	1.91	3.82



EVALUATION

Evaluation Scenario	Key Findings
Cold Start Test	Initial function invocation exhibited higher latency due to cold start initialization, while subsequent warm executions achieved significantly lower and more stable response times.
Load Test	The Azure Function maintained stable response times, consistent throughput, and 0% request failures under sustained workloads of up to 50 concurrent users, demonstrating reliable performance.
Stress Test	The application continued processing requests under increasing user load; however, response times increased as system resources became more heavily utilized.
Spike Test	The Azure Function successfully handled sudden increases in traffic and recovered quickly without request failures, demonstrating resilience under abrupt workload changes.
Payload Size Test	Increasing payload size significantly increased response time and reduced throughput, confirming that payload size has a direct impact on application performance.
Mixed Workload Test	The application processed mixed request sizes successfully, maintaining stable operation while exhibiting performance variations based on request complexity and payload characteristics.

CONCLUSION

- Azure Functions successfully deployed and executed the serverless resume matching application.
- **Load testing** demonstrated stable response times, consistent throughput, and **0% request failures** under sustained concurrent workloads of up to 50 users.
- **Cold start testing** revealed a significant initial latency caused by function initialization, while warm executions achieved much lower response times.
- **Spike testing** showed that the application recovered quickly after a sudden increase in concurrent requests without any request failures.
- **Payload size testing** showed that increasing payload size significantly increased response time and reduced throughput, making payload size a key factor affecting performance.
- **Stress testing** demonstrated that Azure Functions scaled effectively under high concurrency, although latency increased as workloads became more demanding.
- **Pareto analysis** shows that 1.0 GB configuration is technically the *absolute most cost-effective* (3.81 GB-s), but the 2.0 GB configuration (3.82 GB-s) is essentially identical in price while delivering **double the speed**.
- Overall, the experimental results demonstrate that **Azure Functions provide a reliable, scalable, and resilient platform for event-driven applications when workload characteristics and resource allocation are appropriately managed.**

Abstract geometric lines in a light brown color, forming various polygons and overlapping shapes on the left side of the slide.

THANK YOU